Introduction

Adult dataset is a 1994 Census bureau database in the UCI Machine Learning Repository since 1996. It includes demographic data of the individuals in the United States of America like age, work class, education, marital status, race and sex. The target variable of the dataset is a binary categorical column about individual's income less or more than fifty thousand dollars. The dataset has 14 attributes with both numerical and categorical features. It is well-labeled with clear instructions about the data and its features. It provides social insights into whether age, education, gender and marriage status affect income.

The dataset is also used as a benchmark to measure the performance of wide variety of classification models in machine learning. (Vapnik & Cortes, 1995)

For example: the adult dataset is used to benchmark the performance of different support vector machine algorithms such as multi-class SVM (Chih-Wei Hsu, 2002), large-scale linear SVM (Hsieh, 2008), text-categorization with SVM (Joachims, 1998).

1. Data Pre-processing

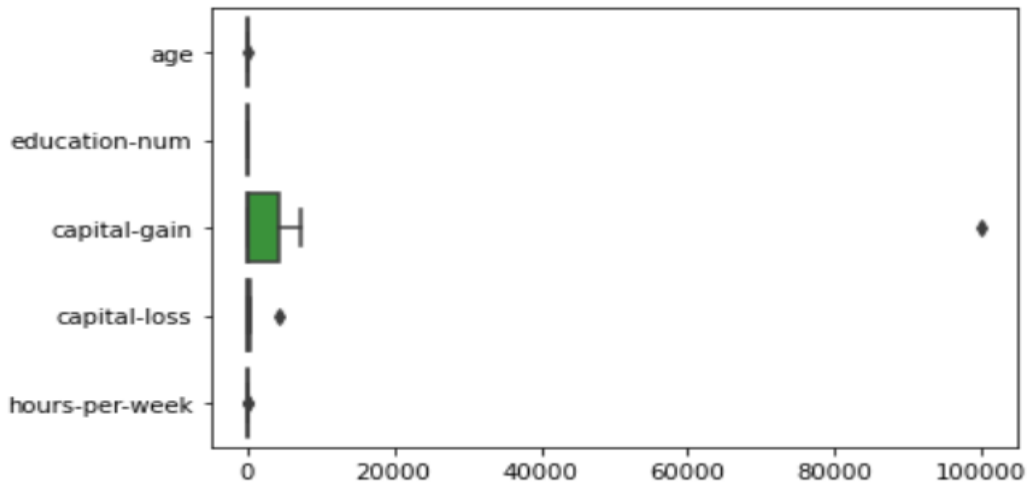
1.1. Duplication removal

The first step in data processing is to remove the duplicates. The removal of duplicates is a significant process because any insights you obtain before duplicate removal can be useless and incorrect. Moreover, during the model training, it can overfit the training data and unable to generalize. It can also cause data leakage from training data to validation data creating biased predictions.

Moreover, it also increases dimension leading to slow training and inference.

We removed 24 duplicate rows. We also removed 'Education' because it was a categorical duplicate of feature 'Education-num'.

1.2. Outliers



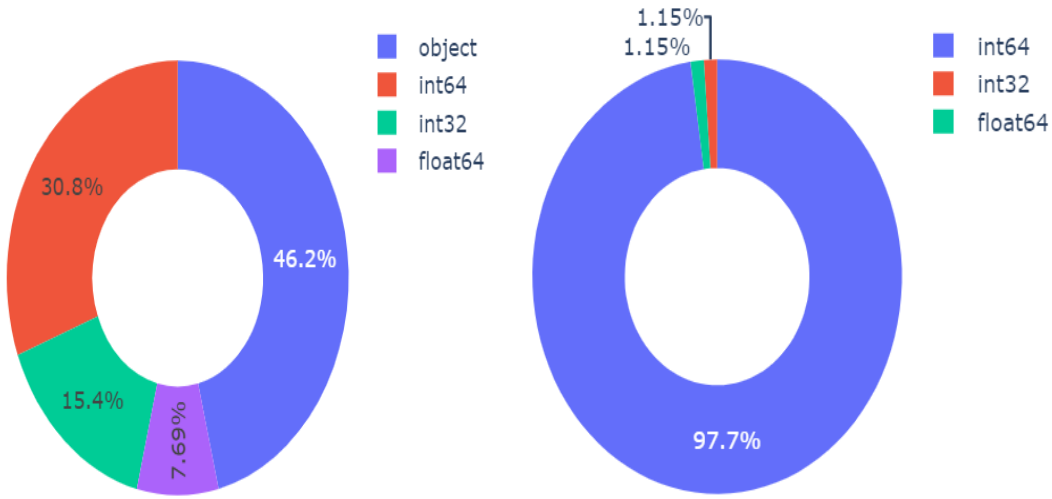
Outliers can distort the overall distribution of data. Moreover, it can create false patterns leading to decreased accuracy, decreased performance, and overfitting/underfitting. (Gupta, Gao, Aggarwal, & Han, 2014)
We discovered 166 outliers in 'capital-gain' and 3 outliers in 'capital-loss' (using q3+1.5*q3-q1 formula). There were outliers in 'fnlwgt' but it was unnecessary to remove them since we removed 'fnlwgt' column later.
The outliers were converted to N/A instead of deleting entire rows.

1.3. Deletion of irrelevant columns

Irrelevant columns include columns that don't help the prediction. We deleted fnlwgt (estimate of how many people belong to this set of combination) because it is not a reliable predictor of income and is correlated with other variables in the dataset. (Kaggle, n.d.)

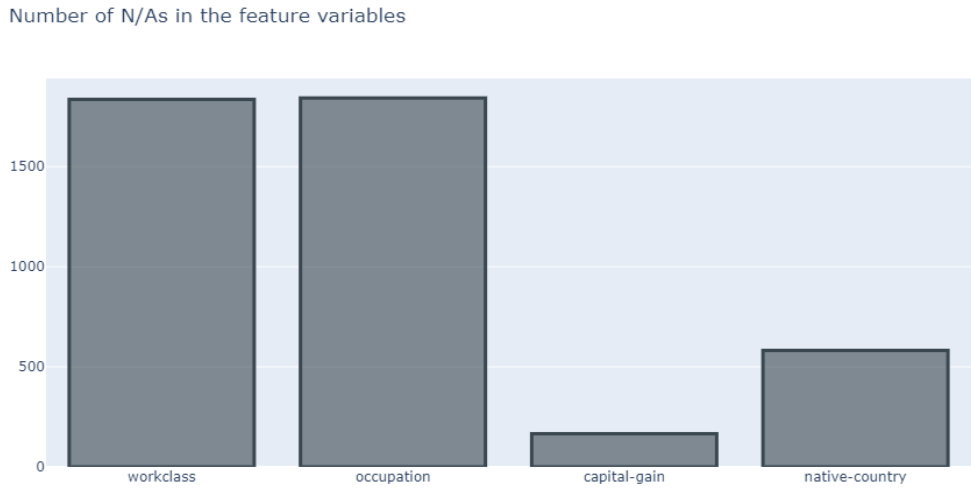
1.4. Converting categorical data to numerical data

The primary way to evaluate success of this process was to convert all features to numerical values such as **float** or **int** when displayed by df.dtypes.
First, we converted the binary categorical data 'sex' to numerical 'gender' using **LabelEncoder**.
Next, we converted other six categorical columns into numerical columns using **OneHotEncoder** and **get_dummies function from Pandas**. Since these columns were not binary, we used each column for unique data values of these categorical data.
For example: If we have column X with unique values a, b, c and d, we created 4 columns with X_a, X_b, X_c, and X_d
This resulted in 87 columns excluding target variable 'salary'. We used this process to retain individuality of each value and to eliminate biasness. Using integer encoding can cause to assume a natural ordering between categories leading to poor performance. (Brownlee, 2020)
The start (left image) and end (right image) result after pre-processing looks like:



1.5. Dealing with missing values

The dataset had missing values in the form of '?' instead of common N/A or null values. The missing values were converted into N/A first. We had N/A values from outlier removal section as well. In total, we have 166 missing values with N/A.



To deal with missing values, we created three datasets:

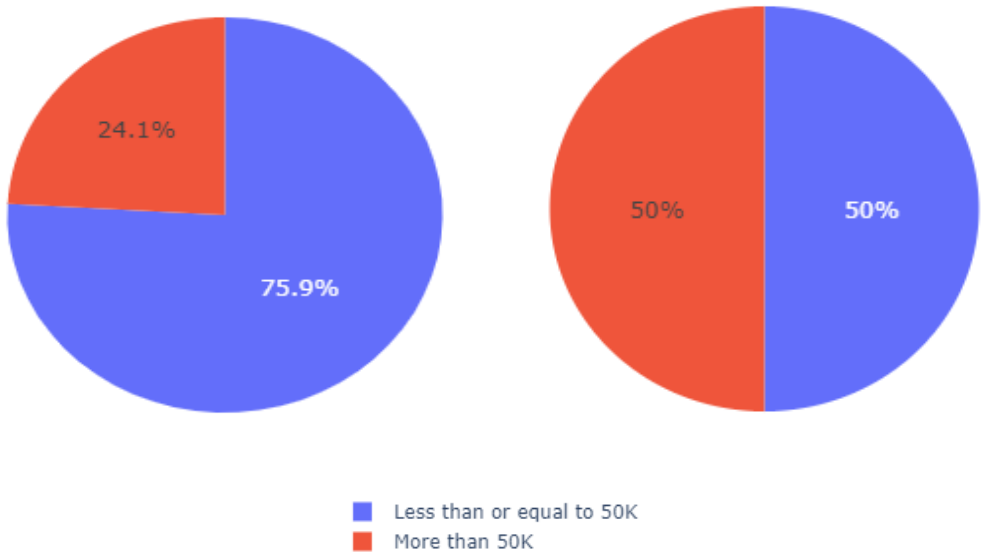
- Remove the rows that contain the missing values (**data_imp_del**). It is the basic and standard way to deal with missing value.
- Fill missing values to median values (**data_imp_med**). More than 10% of data missing and high correlation amongst variables can lead to change in shape of the distribution with decreasing standard deviation. But we only have 166 missing values. So, it is possible to use median as well. (Tsikriktsis, 2005)
- Fill the missing values using 'from sklearn.impute import **KNNImputer**' (**data_imp_knn**). Using KNN imputation can weaken the issue of overfitting of the models. (Zhang, Jin, & Zhu, 2011)

1.6. Data Check for target variable

Imbalance in the target variable can result in poor performance due to biasness towards the majority class. It can also be more difficult for model to identify patterns and converse. It can also impact cross-validation and increases difficulty to model comparison. (Lemaître, Nogueira, & Aridas, 2016)

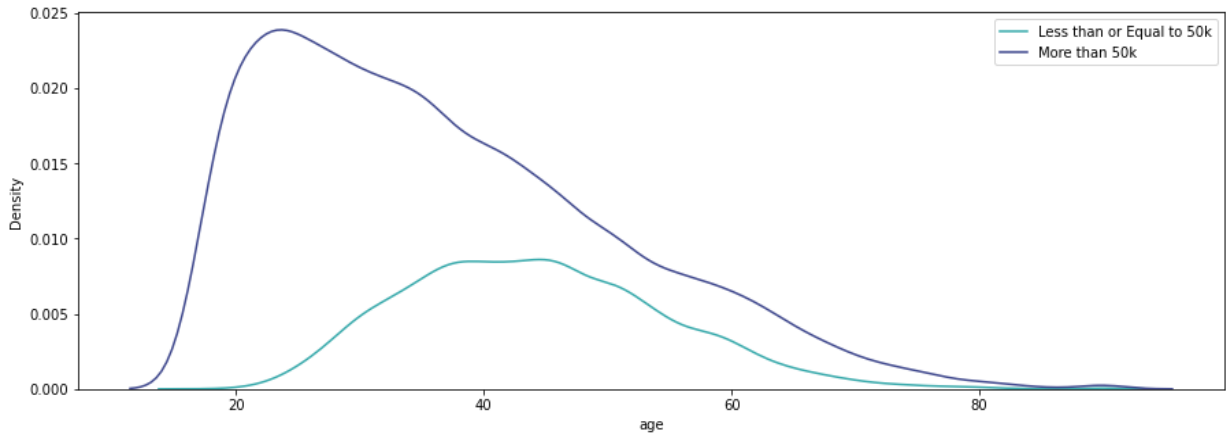
We discovered the proportion ratio was 24.1 to 75.9. Therefore, we used oversampling to balance the dataset using **SMOTE**. **SMOTE** balances the imbalanced dataset by creating synthetic data of minority class (Chawla, Bowyer, Hall, & Kegelmeyer, 2002)**.**

The start (left image) and end (right image) result after target variable distribution looks like:



2. Exploratory Data Analysis:

2.1. Age vs Salary

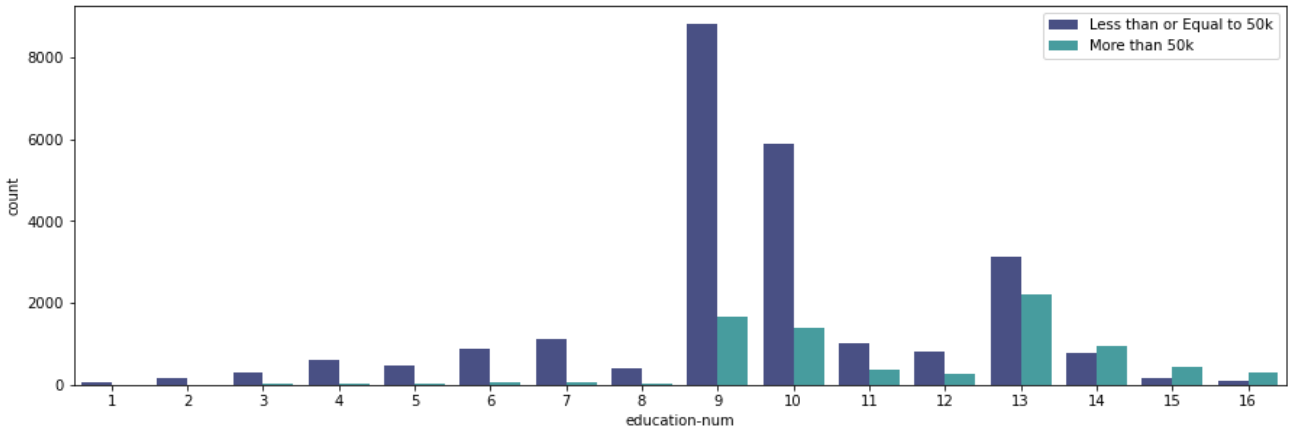


There are two odd elements:

- 19 years old earning more than 50K. It would be possible in today's world as an influencer. But the data was from 1996, which makes this quite strange.
- Age of 61+ still working

Despite these, the data is reasonable. So, we are going to keep the data for now.
There is clear positive correlation. As age increases, so does the salary. This is likely because of work experience, and promotions.

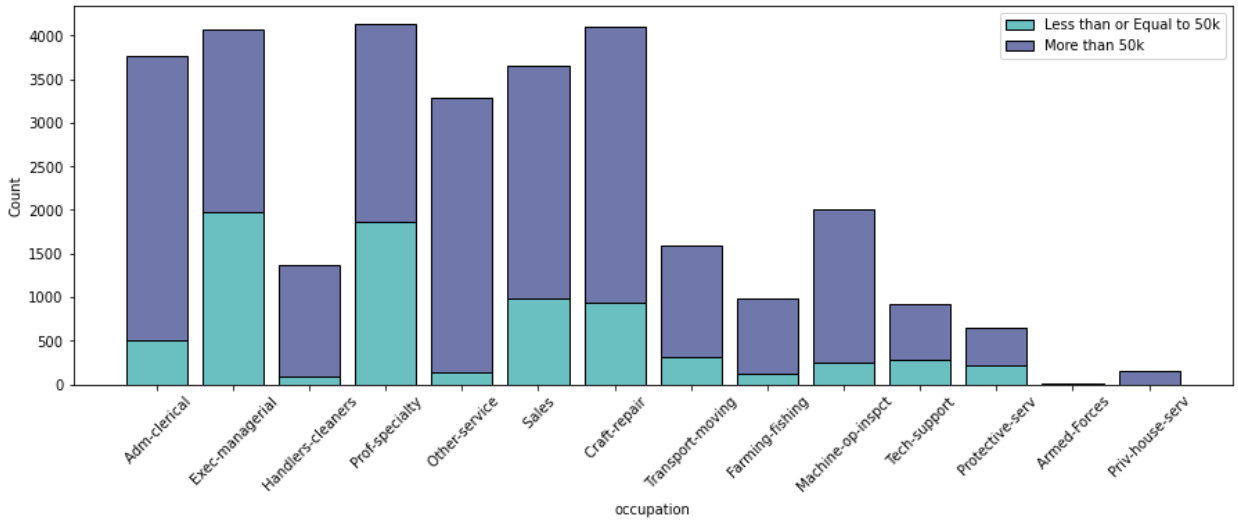
2.2. Education vs Salary



There is a postive relationship but it is not linear. The salary increase between two adjacent levels differ. We can see a diminishing marginal return. Beyond a certain point(14), there is less salary increase compared to previous point. It might be because of the limited nature of high-paying jobs and massive competition.
In a nutshell, the adults wanting to maximize their earning potention should consider a Bachelor's

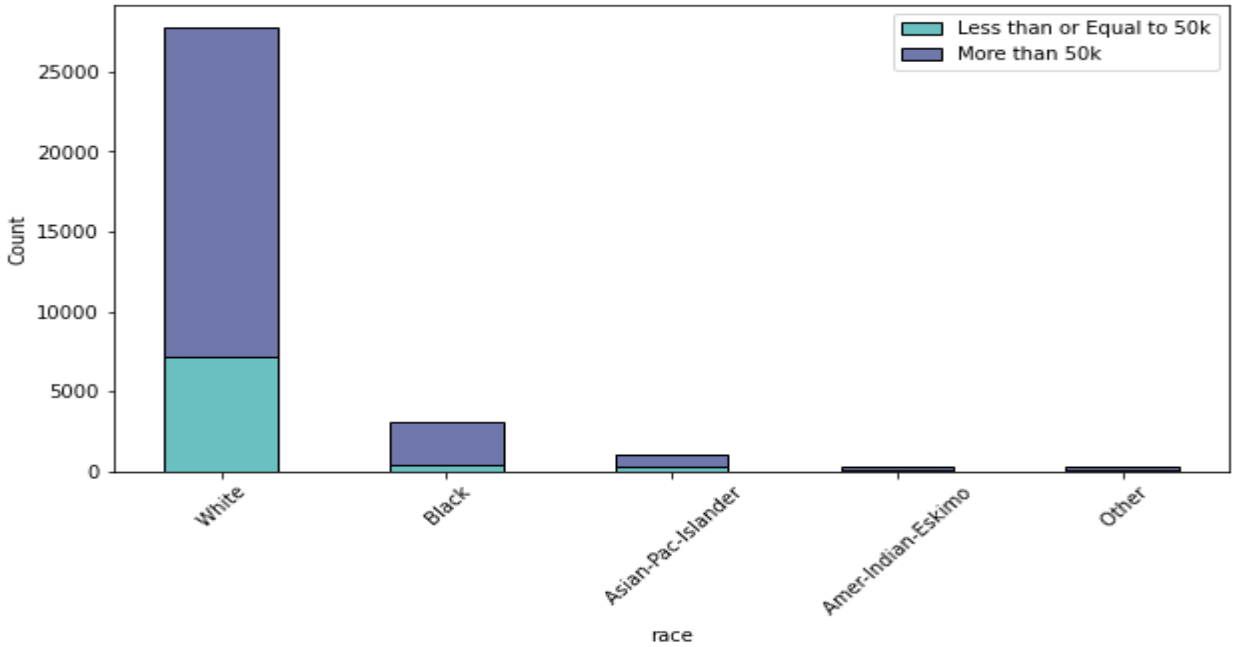
degree(14). There is a need to measure education cost vs salary increase proportion of education beyond that.

2.3. Job Types vs Salary



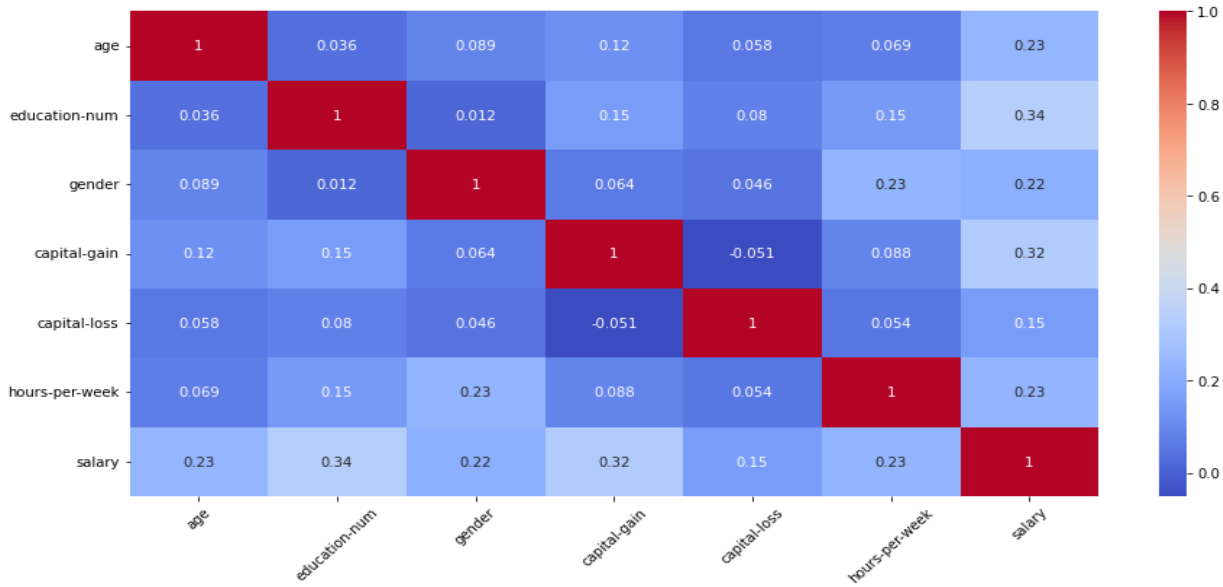
We can observe management and executive positions with significant incomes compared to manual labor positions. It is reasonable as jobs that require more training and responsibility have higher salary. But we observe administration position have surprisingly low salary. Only reasonable explanation is that there is high supply of workforce and less demand during that time.

2.4. Race vs Salary



No clear pattern can be observed expect that fact there are higher numbers of white race compared to other race in the job market.

2.5. Correlation Graph



There seems to be a weak correlation between dependent and independent variables. Therefore, the dataset is not recommended for prediction models.

In conclusion, we cannot expect high accuracy scores for machine learning models with this dataset.

3. Pre-Modelling

3.1. Normalization and PCA

For normalization, we have used **min-max normalization**. Min-max normalization scales all the values in the dataset to be between 0 and 1. This reduces the amount of data to be read and processed. We primarily used Min_max normalization since our data contains outliers and helps to ensure all data is on the same scale, helping to eliminate biasness. It also allows patterns and trends to be easily detected. This helps to increase perfomance and prediction compared to un-normalized dataset.

Normally, normalization creates issue by scaling features with both high and low correlation can interfere with the highly correlated features resulting in deterioration of the performance. (Singh & Singh, 2020)

But, in this dataset, every variable have similar and low correlation, which eliminates such issues.

3.2. PCA

We have also used **PCA** (Principal Component Analysis) to reduce the dimensionality of the data since we have more than 85 columns to process. The n_component of 0.85 in PCA means it explain 85% of the variance in the data.

3.3. Repeated Stratified K Fold application

Repeated Stratified K Fold cross-validates a model splitting the datasets into K folds ensuring each fold are of equally proportioned classes and repeat the process K number of times. (Scikit Learn, 2007-2022)

3.4. Train-test

We split the dataset into proportion of **8:2** for train and test data respectively.

4. Classification with Machine Learning Algorithms

We are using seven classification machine learning algorithms for the dataset. We didn't use **Decision Tree** because they are prone to overfitting. Moreover, we also didn't use **Naïve Bias** because it is not well-adapted for larger datasets with complex structures such as adult dataset. For measuring the accuracy, we use **best_score_** from **GridSearchCV**. The **best_score_** represents the mean cross-validated score of the best estimators.

4.1. Logistic Regression

It is a simple statistical technique used to predict likelihood of an event occurring. We use it because it uses linear model to predict providing a better performance for predicting outcomes. (Scikit Learn, 2007-2022)

4.2. Ridge Classifier

We use it because it can help prevent overfitting by adding a penalty to the loss function. (Scikit Learn, 2007-2022)

4.3. K Neighbors Classifier

It is a non-parametric method and doesn't make any assumption about the data distribution. Adult dataset being complex and large dataset may not fit in any simple parametric models. It also saves time since it doesn't require training set to predict. (Imandoust & Bolandraftar, 2013)

4.4. Support Vector Classifier

We use SVC because it is particularly useful to datasets with a large number of features and with high class imbalance. Adult data has large number of features and class imbalance. But it takes a huge amount of time to train larger datasets.

4.5. Bagging Classifier

We use Bagging Classifier because it can improve the accuracy by reducing the variance and generalizing the model. It prevents model from overfitting. (Scikit Learn, 2007-2022)

4.6. Random Forest Classifier

It can handle large datasets and is resistant to overfitting leading to better predictions.

4.7. Gradient Boosting Classifier

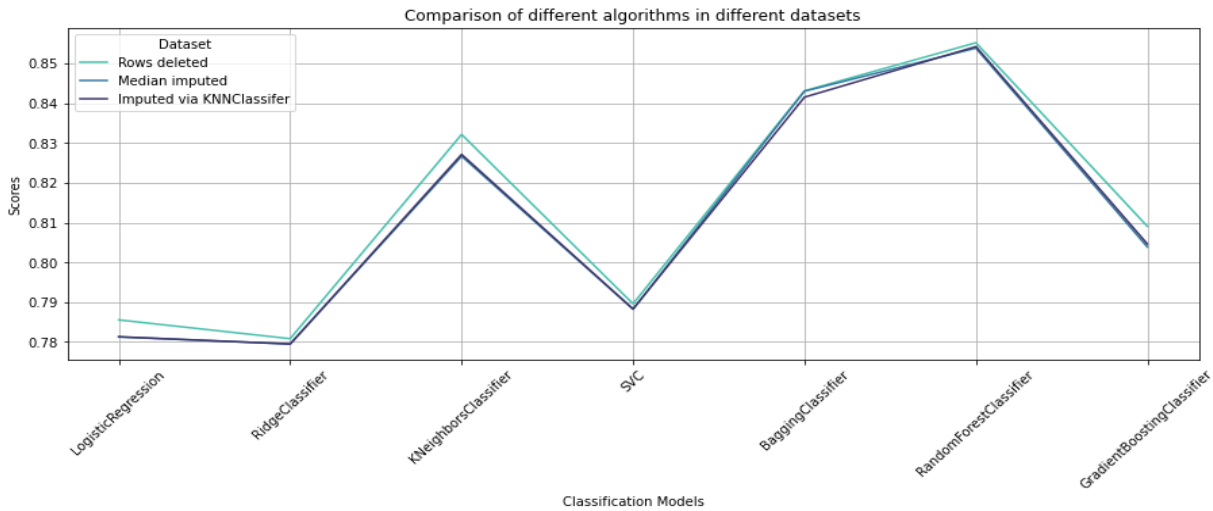
It uses decision tree to identify relationships between variables. It makes it well-suited for predicting datasets with a large number of features like adult dataset. It is a good option for classification with probabilistic outputs. (Scikit Learn, 2007-2022)

4.8. Conclusion

We will look at the results obtained below:

Machine Learning Algorithms (Classification Models)	Output from dataset with deleted rows	Output from dataset with median imputed	Output from dataset with median imputed
Logistic Regression	0.786	0.781	0.781
Ridge Classifier	0.781	0.780	0.780
K Neighbor Classifier	0.832	0.827	0.827
Support Vector Classifier	0.790	0.788	0.788
Bagging Classifier	0.843	0.843	0.841
Random Forest Classifier	0.855	0.854	0.854
Gradient Boosting Classifier	0.809	0.804	0.805

We find all three perform relatively similar with row- deleted dataset performing slightly higher.



5. Hyper tuning the parameters

Since the three datasets gave similar scores, we are going with dataset containing KNN imputer. Each of the algorithms have their own parameters.
So, we use **GridSearchCV** to find best parameters of each algorithm and find the best algorithm suited for the adult dataset.

5.1. Logistic Regression

The parameters we use for logistic regression are:

- **solver**: Algorithm used in optimization.
  Optimum solver is '**lbfgs**'. '**lbfgs'** handles multinomial loss. (Scikit Learn, 2007-2022)
- **penalty**: Regularization. Optimum penalty is '**l2**'.
- **C**: Controls the penalty strength. Optimum C is **1.0**.

5.2. Ridge Classifier

The parameters we use for ridge classifier are:

- **alpha**: Regularization Strength.
  Must be positive. (Scikit Learn, 2007-2022) Optimum alpha is '**20**'.
- **solver**: Used '**sag'**, which uses Stochastic Average Gradient descent. (Scikit Learn, 2007-2022)

5.3. K Neighbors Classifier

The parameters we use for K Neighbor Classifier are:

- **n_neighbors:** Number of neighbors to use for **kneighbors** queries. (Scikit Learn, 2007-2022) Optimum is '1'.
- **weights:** Weights for each point in prediction. Optimum is '**uniform'.**
  It means all points(nodes) in each neighborhood are weighted equally. (Scikit Learn, 2007-2022)
- **metric:** Used to measure the distance. (Scikit Learn, 2007-2022)
  Optimum metric is '**manhattan'.**
- **algorithm:** Algorithm to compute nearest neighbors. (Scikit Learn, 2007-2022)
  Optimum algorithm is **'kd_tree'.**
- **leaf_size:** Optimum leaf size is **20.**
- **p:** Optimum p is **1.**

5.4. Support Vector Classifier

The parameters we use for SVC are:

- **kernel:** Kernel type to be used in the algorithm. Optimum kernel is **'rbf'.**
- **C:** Regularization parameter. Optimum C is **1.0.**
- **gamma:** kernel coefficient for **'rbf'.** We used **'scale'** as parameter.

5.5. Bagging Classifier

The parameters we use for Bagging Classifier are:

- **n_estimators:** Number of base estimators in the ensemble. Optimum is **1000.**
- **max_features:** Optimum is **0.7.** Classifier takes random features of size 0.7 of the original data
- **max_samples:** Optimum is **0.7.** Classifier takes random samples of size 0.7 of the original data

5.6. Random Forest Classifier

The parameters we use for Random Forest Classifier are:

- **criterion:** We use **'gini'** as the only parameter, which uses the Gini impurity to measures the quality of a split.
- **n_estimator:** Optimum n_estimator is **1000**.
- **max_features:** We use **'sqrt'** which causes max_features to be square root of n_features. (Scikit Learn, 2007-2022)
- **min_samples_split:** The optimum min samples split is **2**.
- **min_sample_leaf:** The optimum min sample leaf is **1**.
- **bootstrap:** The optimum bootstrap is **False**, which means entire dataset is used to build each tree. (Scikit Learn, 2007-2022)

5.7. Gradient Boosting Classifier

The parameters we use for Gradient Boosting Classifier are:

- **n_estimator:** Optimum is **1000**.
- **learning rate:** The optimum learning rate is **0.1.**
- **subsample:** The optimum is **0.7**.
  Since, it is smaller than **1.0**, it results in Stochastic Gradient Boosting. (Scikit Learn, 2007-2022)
- **max_depth:** The optimum max depth is **7.**
- **max_features:** We use '**sqrt'** for max features.

5.8. Conclusion

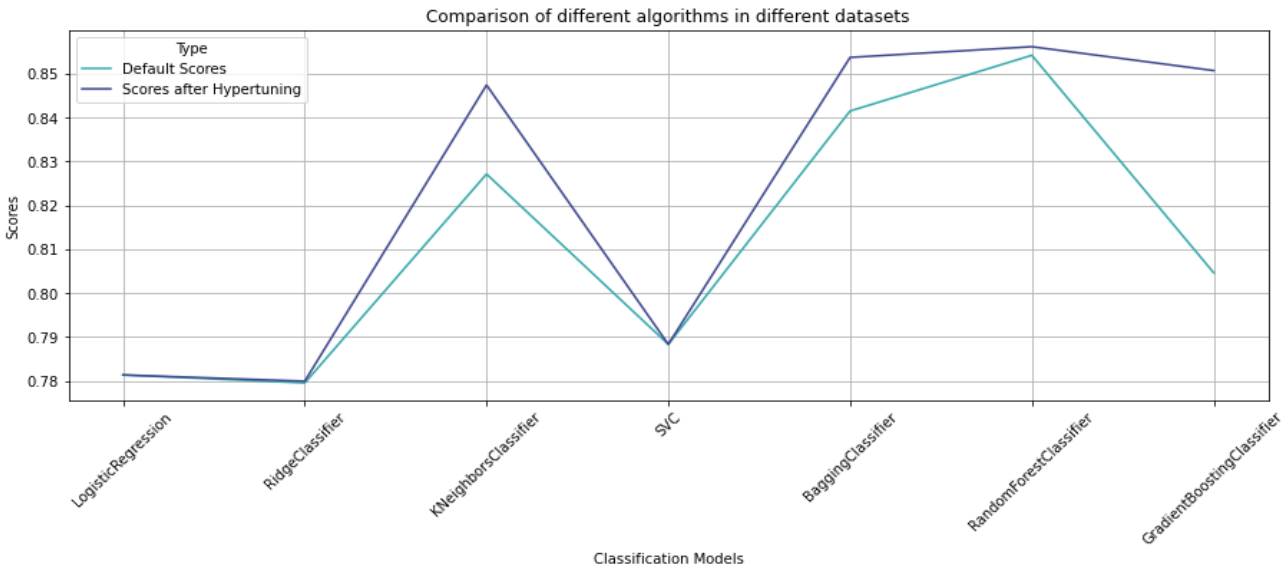
From the table and graph below, we can see that the prediction outcome with hyper tuning is better than the prediction outcome without hyper tuning for all the machine learning algorithms.

The **Logistic Regression**, **Ridge Classifier**, **Support Vector Classifier**, and **Bagging Classifier** all have similar performance without hyper tuning. However, after hyper tuning, the **K Neighbor Classifier** and **Random Forest Classifier** outperform all other models.

The **Gradient Boosting Classifier** has the worst performance without hyper tuning but the best performance with hyper tuning**. It seems the Gradient Boosting Classifier is more sensitive to hyperparameter tuning than the other models.**

In this model, we get **Random Forest Classifier** (**0.856**) as the best machine learning algorithms.

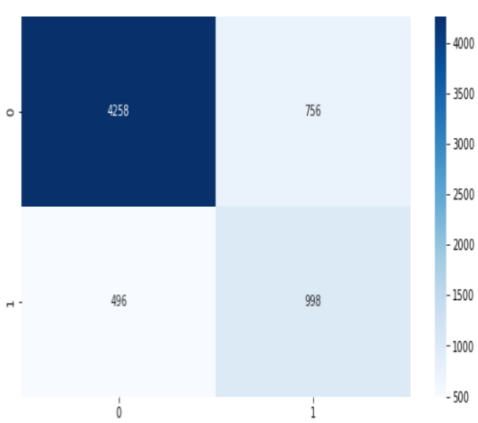
Machine Learning Algorithms (Classification Models)	Prediction Outcome	Prediction Outcome with Hyper tuning
Logistic Regression	0.781	0.781
Ridge Classifier	0.780	0.780
K Neighbor Classifier	0.827	0.847
Support Vector Classifier	0.788	0.788
Bagging Classifier	0.841	0.854
Random Forest Classifier	0.854	0.856
Gradient Boosting Classifier	0.805	0.851



5.9. Extra Stats about best model

We re-ran the model with best parameters. We received **score of 0.864,** which is rather surprising. It might be because of only 3 K-folds and one repeats. But it was a trade made to lessen the time taken for the hyperparameters tuning.

Metrics accuracy is **0.808**, which is quite decent. Afterall, we expected low accuracy because variables in this dataset are of low correlation and not suitable for prediction.




```
Best: 0.864137 with {'bootstrap': False, 'criterion': 'gini', 'max_fe
_estimators': 1000} parameters
0.8076213890596189
                          precision    recall  f1-score   support

Less than or Equal to 50k       0.90      0.85      0.87      5014
            More than 50K       0.57      0.67      0.61      1494

                 accuracy                           0.81      6508
                macro avg       0.73      0.76      0.74      6508
             weighted avg       0.82      0.81      0.81      6508
```

6. Comparison with other works on this dataset

6.1. First Work

This work was extracted from Kaggle. The above table represents the cv mean obtained from the models. (DOUGHNUTTRAP, 2017)

Machine Learning Algorithms (Comparison of Classification Models)	**Other code's outcome**	**Personal Outcome with Hyper tuning**
Logistic Regression	0.840	0.781
Ridge Classifier	*N/A*	0.780
K Neighbor Classifier	0.843	0.847
Support Vector Classifier	*N/A*	0.788
Bagging Classifier	*N/A*	0.854
Random Forest Classifier	0.854	0.856
Gradient Boosting Classifier	0.805	0.851
Naïve Bayes	0.807	*N/A*
Linear SVM	0.841	*N/A*
Radial SVM	0.852	*N/A*
Decision Tree	0.814	*N/A*

The code processes in the two outcomes are quite similar and follow standard processes of **K-fold** and **PCA**. But, one thing that is different is other code uses **fnlwgt** and deletes **native country** and **education** while my code deletes **fnlwgt** only.
But there is no clear winner, as each algorithm performed similarly. The Personal Outcome with Hyper tuning slightly outperforming the other work's outcome.
This is likely due to the other work's outcome using a smaller number of params for hyper tuning.
The results suggest that hyper tuning more parameters can be beneficial in terms of accuracy. Furthermore, the results also suggest that the benefits of hyperparameter tuning may be more pronounced when the model is more complex (**for example**: Random Forest or Gradient Boosting).

Bibliography

Brownlee, J. (2020, 6 30). *Why One-Hot Encode Data in Machine Learning?* Retrieved from Machine Learning Mastery: https://machinelearningmastery.com/why-one-hot-encode-data-in-machine-learning/

Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). Smote: Synthetic Minority Over-Sampling Technique. *Journal of Artificial Intelligence Research, 16*, 321-357.

Chih-Wei Hsu, C.-J. L. (2002). A comparison of methods for multiclass support vector machines. *IEEE Transactions on Neural Networks*, 415-425.

DOUGHNUTTRAP. (2017). *Income Classification Model | Kaggle.* Retrieved from Kaggle: https://www.kaggle.com/code/jieyima/income-classification-model

Dua, D., & Graff, C. (2019). *UCI Machine Learning Repository: Adult Data Set*. Retrieved from UCI Machine Learning Repository: http://archive.ics.uci.edu/ml/datasets/Adult

Geeks For Geeks. (2021, 8 10). *Detect and Remove the Outliers using Python.* Retrieved from Geeks for Geeks: https://www.geeksforgeeks.org/detect-and-remove-the-outliers-using-python/

Gupta, M., Gao, J., Aggarwal, C. C., & Han, J. (2014). Outlier Detection for Temporal Data. *Synthesis Lectures on Data Mining and Knowledge Discovery*, 1-129.

Hsieh, C.-J. a.-W.-J. (2008). A Dual Coordinate Descent Method for Large-Scale Linear SVM. *ICML '08: Proceedings of the 25th international conference on Machine learning*, 408–415.

Imandoust, S. B., & Bolandraftar, M. (2013). Application of K-Nearest Neighbor (KNN) Approach for . *Journal of Engineering Research and Applications*, 605-610.

Joachims, T. (1998). Text categorization with Support Vector Machines: Learning with many relevant features. *Machine Learning: ECML-98*.

Kaggle. (n.d.). *Adult Census Income*. Retrieved from Kaggle: https://www.kaggle.com/datasets/uciml/adult-census-income

Lemaître, G., Nogueira, F., & Aridas, C. K. (2016). Imbalanced-learn: A Python Toolbox to Tackle the Curse of Imbalanced Datasets in Machine Learning. *Journal of Machine Learning Research 7*.

Scikit Learn. (2007-2022). *sklearn.ensemble.BaggingClassifier*. Retrieved from Scikit Learn Documentation: sklearn.ensemble.BaggingClassifier

Scikit Learn. (2007-2022). *sklearn.ensemble.GradientBoostingClassifier*. Retrieved from Scikit Learn Documentation: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.GradientBoostingClassifier.html

Scikit Learn. (2007-2022). *sklearn.ensemble.RandomForestClassifier*. Retrieved from Scikit Learn: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html

Scikit Learn. (2007-2022). *sklearn.linear_model.LogisticRegression*. Retrieved from Scikit Learn Documentation: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html

Scikit Learn. (2007-2022). *sklearn.linear_model.RidgeClassifier*. Retrieved from Scikit Learn Documentation: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.RidgeClassifier.html

Scikit Learn. (2007-2022). *sklearn.model_selection.RepeatedStratifiedKFold¶*. Retrieved from Scikit Learn Documentation: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.RepeatedStratifiedKFold.html

Scikit Learn. (2007-2022). *sklearn.neighbors.KNeighborsClassifier*. Retrieved from Scikit Learn Documentation: https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html

Singh, D., & Singh, B. (2020). Investigating the impact of data normalization on classification performance. *Applied Soft Computing*.

Tsikriktsis, N. (2005). A review of techniques for treating missing data in OM survey research. *Journal of Operations Management*, 53-62.

Vapnik, V., & Cortes, C. (1995). Support-Vector Networks. *Machine Learning*, 273–297.

Zhang, S., Jin, Z., & Zhu, X. (2011). Missing data imputation by utilizing information within incomplete instances. *Journal of Systems and Software*, 452-459.